arXiv:2606.30005v3 [cs.CL] 16 Jul 2026

LLM Agents Are Latent Context Managers: Eliciting Self-Managed Context via State Proprioception

Binyan Xu[1,2,*], **Haitao Li**[2,†], **Kehuan Zhang**[1]
[1]The Chinese University of Hong Kong, [2]LIGHTSPEED
{binyxu, khzhang}@ie.cuhk.edu.hk, 729156675@qq.com
*Work done during an internship at Tencent. †Corresponding author.

Project Page https://vista-agent.github.io/
Code https://github.com/binyxu/VISTA/

Abstract

Long-horizon tool agents are bottlenecked by how their context grows toward the limits of the context window. Recent systems make context management agent- or system-controlled, but they either learn compression policies that discard evidence or manage context in a layer the agent never sees. We argue that both miss a more basic gap: frontier language models are proprioceptively blind to their own context. From the prompt alone they cannot see how large, old, or used each block is, the signals needed for keep-or-drop decisions. We introduce VISTA (Visible Internal State for Tool Agents), a training-free, model-agnostic layer that represents working memory as typed addressable blocks, surfaces a runtime dashboard of token usage, recency, and access history, and archives blocks as recoverable full-fidelity payloads. On LOCA-Bench, BrowseComp-Plus, and GAIA, the same untrained interface transfers across 1M-, 100K-, and 10K-scale trajectories. On LOCA-Bench it lifts Gemini-3-Flash from 22.7 to 50.7%, reaches 58.0% on BrowseComp-Plus, and remains competitive on GAIA. Gains grow with context pressure and transfer across backbones, while ablations confirm that the dashboard matters beyond archive and recovery tools.

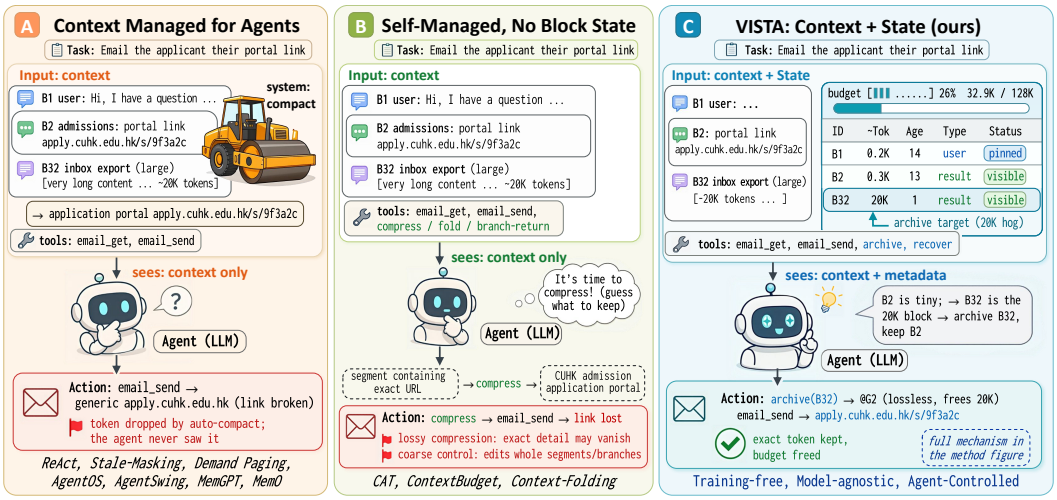


Figure 1: Who manages context, and on what information. Fixed rules compact context the agent cannot see, and blind self-management guesses without state. VISTA surfaces per-block metadata, so the agent archives the large block losslessly.

1 Introduction

Language agents operate over stateful tasks such as filling spreadsheets from web and email evidence, modifying databases, preparing application materials, debugging code, and coordinating business workflows [16, 43]. Their context is working memory. It accumulates tool evidence, stale observations, failed attempts, user constraints, hypotheses, file paths, and action contracts that must remain

correct many steps later [22, 25, 30]. As the task runs, working memory grows until it crowds or overflows the context window, a pressure also studied in long reasoning systems that summarize or carry state across computation [2, 11, 28]. The agent must decide what to keep visible, what to set aside, and what to recover. How this growing context is managed determines whether long-horizon agents succeed.

Existing approaches differ in who makes these decisions. One family keeps the decision outside the agent. Stale-observation masking hides old tool outputs by rule [44], and OS-style layers page or evict context beneath the agent. These layers track statistics such as size, age, and usage, but only inside the runtime. The agent cannot inspect them, and a fixed rule cannot know which evidence will matter later. A second family moves the decision into the agent and learns it from data. Context-as-a-tool fine-tunes a compressor, and budget-aware methods train compression policies with reinforcement learning [16, 24, 25, 30]. These methods can improve performance, but they often discard evidence through summarization or deletion and are tied to the training setting. Across both families, the agent can read block contents but cannot perceive the state needed for a keep-or-archive decision, including block size, recency, access history, and remaining budget. Figure 1 contrasts these families with our approach on one task.

We take an elicitation view. We hypothesize that capable models already contain context-management competence from pretraining on note-taking, retrieval, and reorganization traces, and that the bottleneck is a missing interface rather than a missing policy. Context management is a meta-tool decision over the agent's own working memory, made under partial observability. The agent must choose what to keep or externalize while the prompt omits the runtime state that governs the choice. Learned policies can compensate through training, but this entangles what information should be exposed with what policy should act on it. VISTA isolates the first question by making the state observable without training or model-specific adaptation.

This proprioceptive view implies three requirements. The interface must expose per-block token cost, recency, access history, and remaining budget. It must be reversible, because one-way deletion or summarization can remove evidence needed later. It must be model-agnostic, so gains reflect elicitation through the interface rather than a policy trained for one backbone or domain.

We introduce VISTA (Visible Internal State for Tool Agents), a context layer that represents working memory as typed, addressable blocks and surfaces a dashboard with per-block token usage, recency, access history, and budget. The dashboard is a proprioceptive view of the agent's context state. The agent can archive bulky blocks as external payloads with stable handles and recover exact bytes on demand. Archived payloads are exact transcripts, so removing a block from the prompt does not destroy it. We show both are necessary: recovery, because discarded evidence cannot otherwise be restored; and the dashboard, because a size-blind manager over-archives or under-recovers even given recovery (Proposition 1 and Theorem 1). VISTA requires no training and wraps any backbone.

Empirically, Table 1 tests VISTA across million-, 100K-, and 10K-token trajectories in LOCA-Bench, BrowseComp-Plus, and GAIA. In LOCA-Bench [43], it solves 38/75 tasks versus 17 for ReAct and 32 for Claude Code, with lower trajectory cost than Claude Code. On BrowseComp-Plus, it reaches 58.0% versus 52.0% for the strongest baseline, and remains competitive on GAIA. Pressure sweeps show that the advantage grows on long trajectories, where VISTA cuts active-context overhead while improving accuracy. The same untrained layer improves all four tested backbones, and ablations show that the dashboard matters beyond archive and recovery tools. In transfer, VISTA also reaches higher F1 than the specialized AMA-Bench agent without memory tuning.

This paper makes three contributions.

- We frame context management as a meta-tool decision under partial observability and identify context proprioception as the missing interface, since LLM agents cannot read their own context state from the prompt.
- We introduce VISTA, a training-free context layer whose dashboard exposes per-block metadata and pairs it with lossless archive and recovery. Two matched separations prove both are necessary: recovery to preserve evicted evidence, the dashboard to use it efficiently.
- We show across LOCA-Bench, BrowseComp-Plus, and GAIA that VISTA elicits self-management across trajectory scales and backbones, isolate the dashboard with ablations, and demonstrate transfer on AMA-Bench.

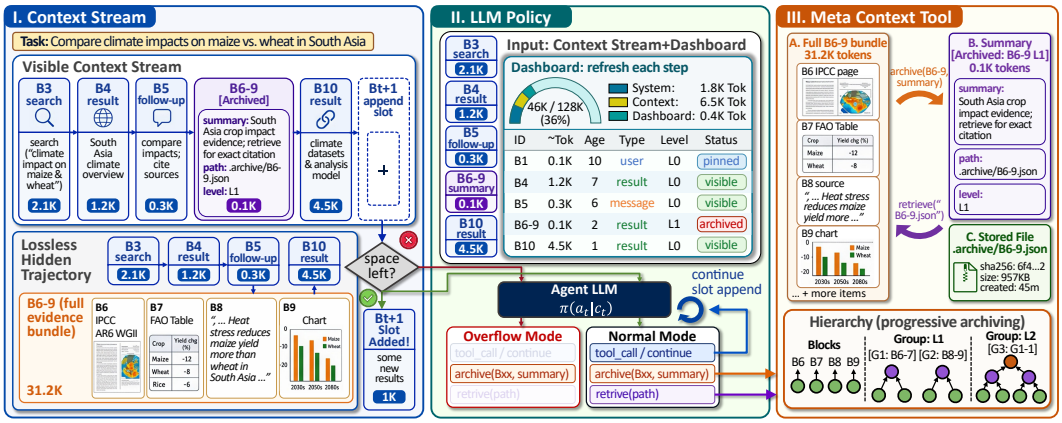


Figure 2: VISTA architecture. Messages and tool outputs become addressable blocks. The dashboard exposes budget and handles to the agent, while archived payloads remain recoverable outside the active prompt.

2 Methodology

VISTA treats context management as a meta-tool decision over the agent's own working memory. Figure 2 shows the three-stage loop: a context stream, a refreshed dashboard, and archive/recovery tools. The goal is to make context state perceptible to an unmodified model without fine-tuning, model-specific changes, or destroyed evidence.

2.1 Problem Setup

At step t, a tool agent has a task goal g, raw interaction history H_t, environment tools $\mathcal{T}_{\text{env}}$, and a context budget B. We write the history as action-observation pairs $H_t = (a_1, o_1, \ldots, a_{t-1}, o_{t-1})$. A standard ReAct-style harness serializes this append-only history into the next model input. Once the serialized history exceeds B, the harness must truncate, clear, mask, or summarize prior content. These interventions conflate what remains visible, what is preserved exactly, and what can be recovered later.

VISTA separates these choices through a workspace $W_t = (V_t, A_t, P_t)$: visible blocks, archived payloads, and blocked large results. The model does not act on the raw transcript directly; it acts on the workspace rendering

$$\begin{aligned} C_t &= \text{render}(V_t) \oplus \text{handles}(A_t) \oplus \text{notices}(P_t) \oplus D_t, \\ a_t &\sim \pi_\theta(\cdot \mid C_t), \qquad |C_t| \le B. \end{aligned} \tag{1}$$

Here D_t is the dashboard: a factual ledger of block IDs, token estimates, recency, access history, archive levels, and remaining budget. The raw transcript is still logged for evaluation, but the agent's working memory is the workspace, not the append-only history.

Workspace invariants. The harness enforces a budget constraint, $|C_t| \le B$, after preflight and final assembly; every actionable unit must have a stable block ID or handle; and archived payload bytes remain recoverable unless the agent explicitly deletes them. The dashboard reports the same token estimates used by these checks, so the agent sees the state that the harness will enforce.

2.2 Context Stream

The left panel of Figure 2 shows how VISTA rewrites the transcript as a block stream. The state space is deliberately small and aligned with the three storage regimes:

(i) VISIBLE blocks appear in the active prompt with exact content; pinned blocks are a protocol-required subset of this state.

(ii) ARCHIVED blocks are replaced by compact handles and summaries, while their original bytes are stored externally.

(iii) BLOCKED blocks are oversized tool results that would exceed the budget if inserted; deleted blocks are intentionally unrecoverable.

This stream gives the agent an address space for context decisions. When a bundle of evidence becomes too large, the stream keeps a compact handle while the exact payload remains in the hidden trajectory. New observations enter only if space remains; otherwise, the workspace enters overflow mode.

Structure preservation. The visible stream is the next-call working set, while the hidden trajectory stores exact payloads that may be needed later. Tool results remain linked to their assistant tool calls, and archived results are rendered in protocol-valid form. If a parent call is also archived, the placeholder becomes ordinary context rather than an orphaned tool response.

2.3 LLM Policy

Dashboard input. The middle panel shows what the model receives: the visible context stream plus a dashboard. The dashboard is regenerated after every tool result is registered, so the agent acts on the current workspace state. It is a ledger over blocks rather than a memory oracle: it exposes runtime state created by the harness or by the agent's earlier actions and does not add hidden task evidence.

Unified action space. The same model policy chooses ordinary environment actions and context actions:

$$\begin{aligned} \mathcal{T}_{\mathrm{ctx}} &= \{\mathrm{archive}(\mathcal{S}, \rho),\ \mathrm{read}(h, q),\ \mathrm{delete}(\mathcal{S})\}, \\ a_t &\in \mathcal{T}_{\mathrm{env}} \cup \mathcal{T}_{\mathrm{ctx}}. \end{aligned} \tag{2}$$

Thus context management is not a separate post-hoc controller. It is part of the model's action space, conditioned on the dashboard and task evidence still visible in C_t.

Mode switch. The dashboard is the proprioceptive channel of the meta-tool. From prompt text alone, a model cannot reliably infer how costly a block is, how recent it is, or whether it has been used again. These signals determine whether a block should stay visible, be archived, or be recovered. In normal mode, the agent may call environment tools, continue the task, archive blocks, or read archived payloads. In overflow mode, ordinary tool calls are disabled until the agent reduces the visible context. The allowed action set is therefore

$$\mathcal{A}_t = \begin{cases} \mathcal{T}_{\mathrm{env}} \cup \mathcal{T}_{\mathrm{ctx}}, & |C_t| \le B, \\ \mathcal{T}_{\mathrm{ctx}}, & |C_t| > B. \end{cases} \tag{3}$$

The hard budget is enforced by the harness, but the agent chooses what to move. A final preflight guard may externalize large raw tool-result blocks near the hard limit to prevent API failure; the policy-facing decision remains with the agent.

Access history. The dashboard also records access history. When an archived payload is read or a block handle is used, the corresponding row can be updated in the next dashboard. This gives the agent a compact trace of active blocks without keeping the full transcript in the prompt.

2.4 Meta Context Tool

Archive interface. The right panel shows the meta-context tool. Archiving takes a selected block set $\mathcal{S}$ and a short replacement summary ρ:

$$\begin{aligned} h &= \mathrm{archive}(\mathcal{S}, \rho), \\ \mathrm{read}(h) &\equiv \mathrm{payload}(\mathcal{S}). \end{aligned} \tag{4}$$

The first line creates a compact handle in the visible stream; the second is the lossless contract. The exact payload moves to the external archive, while h keeps the path, level, size, and checksum metadata visible. Recovery is performed through ordinary file or terminal access to the stored payload path. There is no task-specific retrieval oracle. If the payload is too large, the agent may read bounded chunks or rerun the original source tool with narrower arguments.

The stored payload is a transcript of what the model saw, not a guarantee that the source was complete. If a source response was paginated or truncated, the archive preserves that result exactly and leaves re-querying to the agent.

Hierarchical recovery. Archiving is hierarchical. A first archive level may group several raw blocks into a bundle, as B6-9 does in the figure. Later, groups can themselves be archived into coarser handles when context pressure grows. The visible stream therefore stores a small summary and

Algorithm 1 Self-managed context loop

Require: task block b_{task}, budget B, environment tools $\mathcal{T}_{\text{env}}$, context tools $\mathcal{T}_{\text{ctx}}$
Ensure: final answer and workspace trajectory $W_{1:T}$
Notation: V_t visible stream; A_t archived byte store; P_t blocked payloads; D_t dashboard; C_t^{mgmt} disables task tools

$V_0 \leftarrow \{b_{\text{task}}\}$; $A_0, P_0 \leftarrow \varnothing$
while the task is unfinished **do**
  ▷ **Perceive:** expose runtime context state
  Admit new messages into V_t; oversized payloads enter P_t with visible stubs
  $D_t \leftarrow \text{Dashboard}(V_t, A_t, P_t, B)$
  $C_t \leftarrow \text{Assemble}(V_t, A_t, P_t) \cup \{D_t\}$
  ▷ **Make room:** preserve bytes while shrinking the prompt
  If $|C_t| > \beta B$, choose eligible blocks $\mathcal{S} \subset V_t$ to archive
  Replace $\mathcal{S}$ by handles $h(\mathcal{S}, \rho)$ and store exact payloads in A_t
  ▷ **Act:** management-only under overflow, otherwise normal tool use
  $\mathcal{U}_t \leftarrow \mathcal{T}_{\text{ctx}}$ if $|C_t| > B$; else $\mathcal{T}_{\text{env}} \cup \mathcal{T}_{\text{ctx}}$
  $a_t \leftarrow \text{LLM}(C_t^{\text{mgmt}}, \mathcal{U}_t)$ if $|C_t| > B$; else $\text{LLM}(C_t, \mathcal{U}_t)$
  If $a_t = \text{read}(h)$, recover the exact payload and add it to V_t
  If $a_t = \text{archive}(\mathcal{S}, \rho)$, update handles and A_t as above
  If $a_t \in \mathcal{T}_{\text{env}}$, execute it and admit the result
  If $a_t = \text{answer}$, **return** final answer
  $W_{t+1} \leftarrow (V_t, A_t, P_t)$
end while

retrieval guide, while the hidden trajectory stores the exact evidence. This is why VISTA differs from summarization. Summaries guide navigation, but they are not the only representation of the evidence.

Recovery follows the hierarchy in reverse. The agent may inspect a coarse handle, recover the payload, and then decide whether a narrower piece of evidence should return to active context. It need not reload a long transcript when one row or identifier is needed. It can recover the file, search or read a bounded part, and continue with a smaller block.

2.5 Theory: Recovery and Proprioception Are Both Necessary

VISTA couples two mechanisms: lossless archive/recovery and a proprioceptive dashboard. We show that these mechanisms are necessary for two *distinct* reasons, and that neither substitutes for the other. A single instrument drives both arguments: *Fano's inequality*, relating the information an agent holds to the error it cannot avoid. We apply it in two modes. In its classical form it bounds how faithfully a budget-limited prompt can *reconstruct* an evicted block, giving recovery necessity (Proposition 1). In its list form it bounds how sharply a rate-limited interface can *localize* the block worth evicting, giving proprioception necessity (Theorem 1); the only extra step is to measure how many bits each interface supplies, which for a dashboard-free agent is a short channel calculation (Proposition 2). Reconstruction and localization are the two faces of one inequality, and they map directly onto the two component ablations in Section 4: removing recovery and removing the dashboard each disable exactly one face. We isolate each mechanism on the task family that stresses it, so the separations are clean rather than average-case; this is exactly the regime our ablations reproduce. Full proofs are in Appendix A.

Recovery is necessary under budget pressure. Consider a task family $\mathcal{T}_{N,k}$ in which the history contains N evidence blocks $X_1, \ldots, X_N$, each an independent string of k uniformly random bits. These blocks model exact identifiers, table rows, links, or file snippets. The prompt admits at most B bits and $Nk > B$, so the blocks cannot all remain visible. At the final step, the task reveals an index $i^\star$ drawn uniformly and independently of the blocks, and requires the agent to emit $X_{i^\star}$ exactly.

A recovering method may move blocks to external storage and reload the required block after $i^\star$ is known. A non-recovering method keeps only an in-prompt representation R of size at most B bits and cannot reload discarded content. Deletion, masking, summarization, and skeleton compression are non-recovering. Here Fano acts in its *reconstruction* mode: a state of entropy $\leq B$ can faithfully

reproduce only about B/k of the blocks, so an unknown query is likely to land on one it has already lost.

Proposition 1 (Recovery is necessary under budget pressure)**.** *For any non-recovering method whose pre-reveal in-prompt state R satisfies $H(R) \le B$,*

$$\Pr[\textit{correct on } \mathcal{T}_{N,k}] \le \frac{B}{Nk} + \frac{1}{k}.$$

VISTA is correct with probability 1 *whenever the instruction, the N handles, and one recovered block fit within B. In particular, take any regime in which VISTA stays feasible while the raw evidence dwarfs the budget, for instance $k = \lceil \sqrt{N} \rceil$ and $B = cN \log_2 N$ with a constant c large enough to hold the handles, so that $Nk/B \to \infty$. There the lossy bound tends to* 0 *while VISTA stays at* 1*, and the success gap tends to* 1 *as $N \to \infty$.*

The intuition is that a bounded lossy representation retains exact bits for only about B/k blocks; when the future query is unknown it must discard evidence it may later need. VISTA does not reduce the information in archived blocks. It relocates them at handle cost $O(N \log N)$ and pays byte recovery only for the evidence that matters. Choosing *which* block to recover is exactly where the same inequality strikes again, now in list form.

Proprioception is necessary to use recovery efficiently. Recovery removes the losslessness barrier but not the *control* problem: the agent must still decide *which* blocks to move. We therefore give both compared agents the full lossless archive and recovery tools and isolate a single question, namely how sharply the interface can point at the block worth evicting. The answer will turn the binary "dashboard vs. no dashboard" contrast into one continuous law, with the two ablations as its endpoints and Fano's list form as the only tool.

Definition 1 (Make-room instance $\mathcal{M}_{n,L,\ell}$)**.** The working set holds n blocks. A hidden index $J^\star$, uniform on $\{1, \dots, n\}$, marks one *bulky* block of size L; the other $n-1$ *load-bearing* blocks each have size $\ell < L$, with $\kappa := L/\ell \ge 2$ an integer and $\kappa \le n-1$. The prompt is over budget by exactly L tokens, so the harness disables task tools until the agent has archived a set with total size $\ge L$. The bulky block is not needed again; every load-bearing block is queried later and, if archived, must be recovered (one round-trip) to answer correctly. After each archive the agent observes only the *binary* overflow flag, never numeric freed sizes.

Making room *is* localization. To exit overflow the agent archives blocks in some order until the freed size reaches L. Let τ be the position at which $J^\star$ is archived: archiving $J^\star$ frees L at once, while each load-bearing block frees only ℓ, so κ of them are needed otherwise. The agent stops the moment it hits $J^\star$ or has freed L the slow way, so the number of archived load-bearing blocks, which are exactly the ones that must later be recovered, is

$$Z \;=\; \min(\tau - 1,\; \kappa). \tag{5}$$

Thus every recovery round-trip is a load-bearing block the agent archived *before* finding $J^\star$: making room efficiently is exactly the problem of ranking $J^\star$ early. The only lever on that ranking is how much the interface reveals about which block is bulky.

Definition 2 (Proprioceptive interface of rate I)**.** Before acting, the agent receives an observation Y and archives in some Y-measurable order, using private randomness $U \perp (J^\star, Y)$. The *rate* of the interface is the information it carries about the bulky block, $I := I(J^\star; Y)$ (in bits). Two endpoints anchor the scale. The *size-aware* agent (full ledger) observes the exact vector $(s_1, \dots, s_n)$, which pins down the unique block of size L, so $I = \log_2 n$. The *dashboard-free* agent perceives size only through content: it observes $W_j = \ln s_j + \varepsilon_j$ for each block, with i.i.d. noise $\varepsilon_j \sim \mathcal{N}(0, \sigma^2)$, where σ is the model's relative size-estimation error (Appendix H, median 0.43–0.84). Its rate is $I = I_{\text{content}}$, located next.

Proposition 2 (The dashboard-free endpoint is n-independent)**.** *For the content channel of Def. 2,*

$$I_{\text{content}} \;=\; I(J^\star; W) \;\le\; \frac{(\log_2 \kappa)^2}{\sigma^2} \ln 2 \textit{ bits}.$$

*A grossly oversized block is genuinely visible, so content is informative, yet its rate scales with the log size-*ratio $\log_2 \kappa$*, not the* number *of blocks. Since ranking $J^\star$ among n candidates is worth up to $\log_2 n$ bits, for any fixed perception quality σ the content channel sits at a bounded rate while the full ledger sits at $\log_2 n$, so the two ablations are two points on one interface curve rather than two different worlds. (Proof: Appendix B.)*

The recovery cost is now governed by a single quantity, the rate I, and one inequality converts rate into cost. We state it and then the law it yields.

Lemma 1 (Fano, list form)**.** *Let $J^\star$ be uniform on $\{1,\dots,n\}$ and let $\mathcal{L}(Y,U)$ be any (Y,U)-measurable list of size m. With $P_c = \Pr[J^\star \in \mathcal{L}]$,*

$$\log_2 n - I \;=\; H(J^\star \mid Y,U) \;\le\; 1 + P_c \log_2 m + (1-P_c)\log_2(n-m).$$

Theorem 1 (Information–recovery tradeoff)**.** *Let $\nu := \log_2 \frac{n-\kappa}{\kappa} = (1+o(1))\log_2 \frac{n}{\kappa}$. On $\mathcal{M}_{n,L,\ell}$ under an interface of rate I (Def. 2), the expected number of recovered load-bearing blocks obeys*

$$\mathbb{E}[Z] \;\ge\; \kappa\left(1 - \frac{I+1}{\nu}\right).$$

*Consequently, along the single interface curve of rate I: **(i, endpoints)** the size-aware agent ($I = \log_2 n$) archives $\{J^\star\}$ and attains $Z = 0$, hence 0 recoveries and 0 recall errors; the dashboard-free agent sits at $I = I_{\mathrm{content}}$, bounded in n by Prop. 2, so $\mathbb{E}[Z] \ge \kappa\big(1 - \frac{I_{\mathrm{content}}+1}{\nu}\big) \to \kappa$ as $n/\kappa \to \infty$ (the numerator stays bounded while $\nu \to \infty$). **(ii, threshold)** holding $\mathbb{E}[Z] \le \delta\kappa$ requires $I \ge (1-\delta)\nu - 1$ bits. **(iii, price)** the floor is affine in I with slope $-\kappa/\nu$, so saving Δ round-trips costs at least $\Delta\nu/\kappa$ bits of proprioception. **(iv, achievability)** an I-bit bucket ledger with uniform probing inside the indicated bucket attains $\mathbb{E}[Z] \le \frac{1}{2}(n\,2^{-I} - 1)$, so $I \ge \log_2 \frac{n}{2\delta\kappa+1}$ bits* suffice *for $\mathbb{E}[Z] \le \delta\kappa$, and $Z = 0$ at $I = \log_2 n$.*

Because the necessary rate $(1-\delta)\nu - 1$ of (ii) and the sufficient rate $\log_2 \frac{n}{2\delta\kappa+1}$ of (iv) both equal $(1+o(1))\log_2(n/\kappa)$, the law is two-sided: the recovery cost falls sharply from $\Theta(\kappa)$ to $o(\kappa)$ at $I^\star = (1+o(1))\log_2(n/\kappa)$ bits, the point where the interface finally localizes the bulky block to a κ/n fraction of the workspace and recovery tools stop being wasted. The dashboard-free agent reads a genuine, noisy size signal, which Prop. 2 places at a bounded rate; it therefore lands on the costly side of $I^\star$ once the workspace is large. The ledger is necessary not because content hides size, but because content localizes $J^\star$ only to a κ-scale confusion set while a large workspace demands n-scale localization. The empirical signature follows: with a ledger the agent pinpoints the block to evict and archives narrowly; without it it over-archives yet retrieves far less ($255/57$ vs. $69/105$ archive/retrieve events, Figure 6b), and the proxy for I_{content} is the model's size-*ranking* accuracy, which the dashboard drives toward the $I = \log_2 n$ endpoint (Appendix H). Both results are one inequality read two ways: Fano in reconstruction mode makes recovery necessary (Proposition 1); Fano in list mode makes recovery tools useless below $\Theta(\log(n/\kappa))$ bits of proprioception and prices the exchange between bits observed and round-trips saved (Theorem 1). Proofs are in Appendix C.

3 Experiments

3.1 Experiment Setup

Benchmarks. We evaluate across three online regimes with different trajectory scales: LOCA-Bench [43] as the primary million-token stress test, BrowseComp-Plus [3] as a 100K-scale deep-research retrieval transfer, and GAIA [21] on a fixed 165-question validation subset as a shorter general-assistant setting. We additionally use AMA-Bench as a long-memory generalization benchmark: completed agent histories are replayed through the VISTA workspace before question answering, testing whether the same mechanism can operate as trajectory memory. LOCA-Bench is external to this work; we adopt its public 75-configuration suite and scoring protocol, counting errors and timeouts as failures. Full benchmark protocols, subsets, scoring rules, and budget settings are in the evaluation-details appendix.

Baselines and configuration. We compare against fixed external policies, agent-mediated compression, and production-agent baselines. The fixed-policy group includes ReAct, Tool-result Clearing, and stale-observation masking [44]. The agent-mediated group includes SLIM [40], Active Context Compression [25], and a structured Skeleton Compression baseline following context-as-a-tool compressors [16]. We also include Context-Folding [24], Auto-Archive + Recover, and Claude Code (CLI release May 6, 2026). On AMA-Bench, we compare with the benchmark AMA agent and retrieval-style memory baselines. Appendix Table 4 and the implementation appendix give the full capability matrix, prompts, flags, dashboard format, and tool definitions. Learned context managers

Method	Mechanism			LOCA-Bench		BrowseComp-Plus		GAIA	
	State	Ctrl	Recov	Acc↑	Traj↓	Acc↑	Traj↓	Acc↑	Traj↓
Fixed external policy									
ReAct	✗	✗	✗	22.7	3.51M	39.3	163K	61.2	23K
Tool-result Clearing	✗	✗	✗	26.7	2.60M	42.7	161K	65.5	24K
Stale-obs. Masking	✗	✗	✗	28.0	3.32M	38.0	112K	61.8	24K
Skeleton Compression	✗	✗	✗	33.3	2.84M	40.0	139K	70.3	28K
Agent-mediated / lossy									
SLIM (summary)	✗	∼	✗	29.3	3.76M	49.3	162K	67.9	30K
Active Ctx. Compression	✗	✓	✗	36.0	3.20M	42.7	162K	71.5	39K
Context-Folding	✗	✓	✗	34.7	3.41M	43.3	166K	64.8	39K
Lossless external store									
Auto-Archive + Recover	✗	✗	✓	44.0	2.73M	45.3	133K	63.6	20K
Claude Code	✗	✓	∼	42.7	6.72M	<u>52.0</u>	247K	**73.9**	44K
Ours and ablations									
VISTA w/o dashboard	✗	✓	✓	37.3	5.25M	50.0	423K	68.5	24K
VISTA w/o recovery	✓	✓	✗	<u>45.3</u>	2.99M	43.3	161K	72.1	28K
VISTA (full)	✓	✓	✓	**50.7**	2.86M	**58.0**	135K	<u>73.3</u>	33K

Table 1: Main results across scales. The three benchmarks span million-, 100K-, and 10K-token trajectories, testing whether context management transfers across operating regimes. The pattern supports our central claim: long-horizon gains require not just compression or storage, but an agent-visible state interface paired with controllable, lossless recovery.

whose released artifacts do not match this setting are discussed as complementary systems rather than treated as direct empirical rankings.

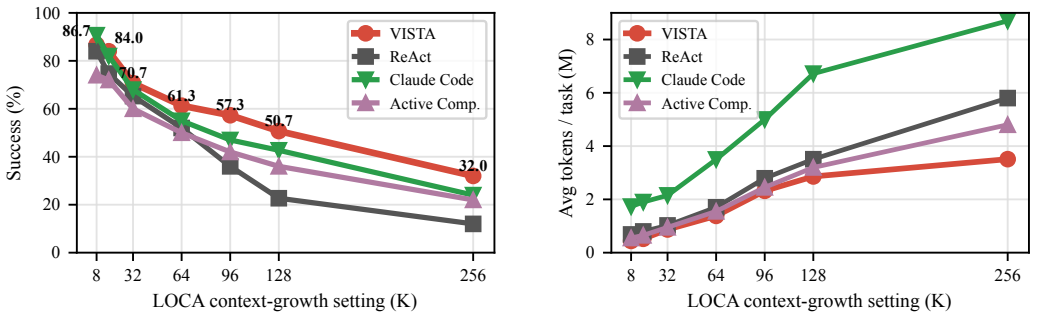
VISTA is training-free and uses the same strategy across backbones. On LOCA-Bench, the main runs use Gemini-3-Flash with a 128K budget. At each turn, the agent sees per-block context metadata and may archive or recover exact transcript payloads; the task tools are unchanged. Across all LOCA-Bench baselines we hold fixed the agent loop, task tools, backbone, context budget, prompt assembly, and scoring; only the context-management policy changes. SLIM and Active Context Compression are reproduced as training-free inference-time baselines, while Skeleton Compression is a structured compression baseline inspired by context-as-a-tool compressors rather than a trained CAT policy. VISTA uses no task-specific retrieval oracle: archived payloads are stored as exact transcripts, and recovery is performed through ordinary file or terminal reads from the returned archive path.

3.2 Main Results Across Scales

Table 1 compares VISTA across million-token LOCA-Bench trajectories, 100K-scale BrowseComp-Plus retrieval, and shorter GAIA trajectories. On LOCA-Bench, VISTA solves 50.7% of tasks, versus 22.7% for ReAct and 42.7% for Claude Code, while using less trajectory than Claude Code. On BrowseComp-Plus it reaches 58.0%, above the strongest baseline at 52.0%. On GAIA it remains competitive in the shorter setting, reaching 73.3% versus 73.9% for Claude Code. The mechanism columns separate state visibility, decision maker, and recovery; methods missing one of these pieces do not match the full interface consistently. The gain is largest in the settings where evidence must survive long trajectories: LOCA-Bench stresses repeated tool interaction, while BrowseComp-Plus stresses retrieval followed by delayed synthesis. GAIA is shorter, so the interface mainly avoids hurting a strong agent rather than opening the larger long-context gap. The LOCA-Bench cost ledger also rules out a spend-more explanation: VISTA uses 2.86M tokens and 36.4 steps per task, compared with 6.72M tokens and 171.5 steps for Claude Code (Appendix Table 5).

3.3 Pressure Regimes

LOCA-Bench million-token pressure. LOCA-Bench creates the failure mode VISTA targets: useful observations arrive early, bulky tool results accumulate, and the agent must still act correctly many steps later. Figure 3 sweeps 8K to 256K context growth on the full 75-task LOCA-Bench suite; exact counts are in Appendix Table 7. The methods are close at low pressure, but the gap



(a) Success vs. context-growth setting **(b) Token cost vs. context-growth setting**

Figure 3: Pressure sweep. Across 8K–256K context growth, VISTA degrades more gracefully than ReAct; the right panel reports average API tokens per task.

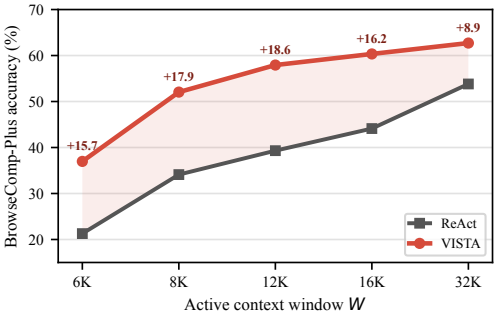


Figure 4: BrowseComp-Plus window sweep. With task set fixed, VISTA peaks at intermediate windows: tiny windows make management overhead costly, while large windows make ReAct truncation less damaging.

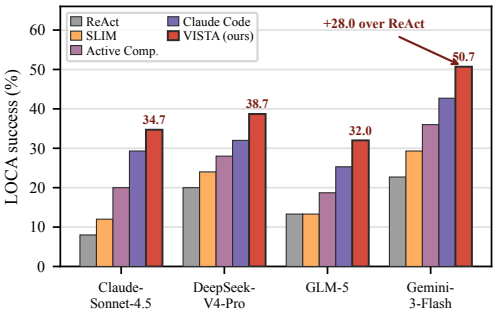


Figure 5: Cross-backbone results. The same untrained VISTA layer is best on all four backbones against ReAct, SLIM, Active Context Compression, and Claude Code.

opens as distractor volume grows: one-way truncation degrades while recoverable externalization remains usable. At 8K the methods are essentially tied (82.7 versus 84.0), but by 128K the gap is 50.7 versus 22.7, and VISTA also spends fewer average tokens (2.86M versus 3.51M). This is the expected signature of recoverable working memory.

BrowseComp-Plus retrieval pressure. BrowseComp-Plus tests whether the interface helps outside the workflow-heavy LOCA-Bench setting. Here the bottleneck is whether early retrieved evidence survives until synthesis. We set $W{=}12$K below the median first-retrieval depth of the gold document ($\approx$ 17K tokens), so ReAct often loses early evidence while VISTA continues under budget. This is a transfer result rather than a leaderboard setting: loose windows usually keep the decisive document visible, so the methods are much closer. Figure 4 varies only W: tiny windows make dashboard overhead costly, large windows let ReAct retain enough evidence, and the gain peaks in the middle. VISTA also uses less active context than ReAct, though more cumulative API tokens, because it survives longer and issues more retrieval rounds.

3.4 Backbone Robustness

The cross-backbone result asks whether the effect is tied to one model family. It is not: the same untrained VISTA layer improves Claude-Sonnet-4.5, DeepSeek-V4-Pro, GLM-5, and Gemini-3-Flash at 128K (Figure 5), including stronger backbones. This supports the elicitation view: capable models can use a context-management interface when runtime state is visible.

3.5 Offline Trajectory-Memory Transfer

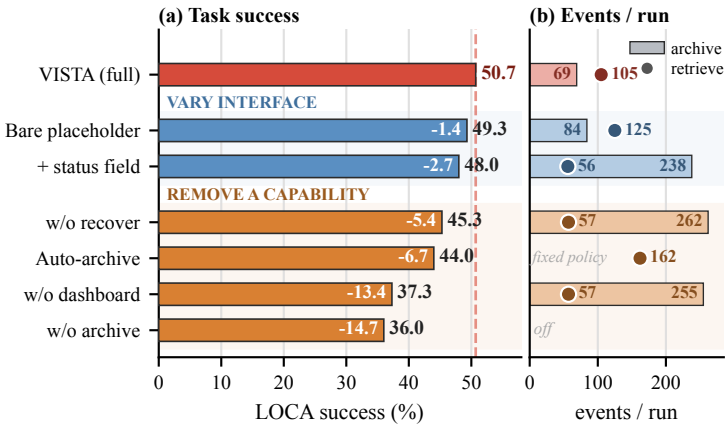


Figure 6: Component ablations. (a) Removing archive, dashboard, recovery, or agent choice hurts more than interface variants. **(b)** Per-run archive vs. retrieve events (same grouping): without the dashboard the agent over-archives yet retrieves far less (255/57 vs. 69/105 for the full system), archiving blindly rather than selectively.

AMA-Bench removes live tool interaction but preserves the long-memory demand: the model must answer questions about completed agent histories. We adapt VISTA by replaying each trajectory into the workspace before QA, so actions and observations become managed blocks rather than a flat prompt. This tests whether VISTA generalizes beyond online context control to a standard long-memory benchmark. On the full 208-episode comparison, VISTA leads on F1, stays within about two points of the specialized AMA agent on judge accuracy, and does so at roughly a quarter of the per-episode runtime; it also outperforms BM25, EMem, and Mem0 adapters on F1. A training-free layer thus stays on par with a purpose-built memory agent while running far cheaper, a clean case of transfer to offline trajectory memory. Table 8 in Appendix gives the by-domain breakdown of AMA-Bench.

Method	F1	Acc.	Runtime/ep	Tokens/ep
BM25	0.335	0.575	**35.5s**	303.43K
EMem	0.363	0.651	166.2s	470.30K
Mem0	0.329	0.536	108.0s	**30.63K**
AMA	0.368	**0.753**	176.5s	268.98K
VISTA	**0.382**	0.731	43.7s	148.49K

Table 2: VISTA as replayed trajectory memory on AMA-Bench. Long-memory evaluation against the specialized AMA agent and memory-style adapters.

3.6 Mechanism Ablations

We probe the method along seven variants that hold the model, task set, token estimator, prompt assembly, and context limit fixed, changing only one mechanism at a time (Figure 6). Four of them remove a capability. No-archive drops recoverable externalization, no-dashboard removes the workspace map, no-recover hides payload paths, and fixed-archive replaces agent choice with a static rule. Two further variants leave every capability intact and perturb only the interface, one rephrasing the archive wording and one rendering the same state as a status board. Because capability and description move on separate axes, any remaining gap is attributable to the mechanism itself rather than to surface wording.

The two axes come apart cleanly. Removing a capability is costly, with no-archive falling to $27/75$ and no-dashboard to $28/75$, while the wording and status-board variants stay within a point or two of the full $38/75$. The gain therefore rides the capability pathway rather than phrasing. Tools alone are not enough either. No-dashboard keeps archive and recovery actions available, yet lacking the state to target them it issues 255 archives against only 57 recoveries, where the full system spends 69 archives and 105 recoveries (Figure 6b). This over-archive, under-retrieve pattern is blind offloading rather than selective management, and it is precisely the behavior the rate-limited interface of Theorem 1 predicts.

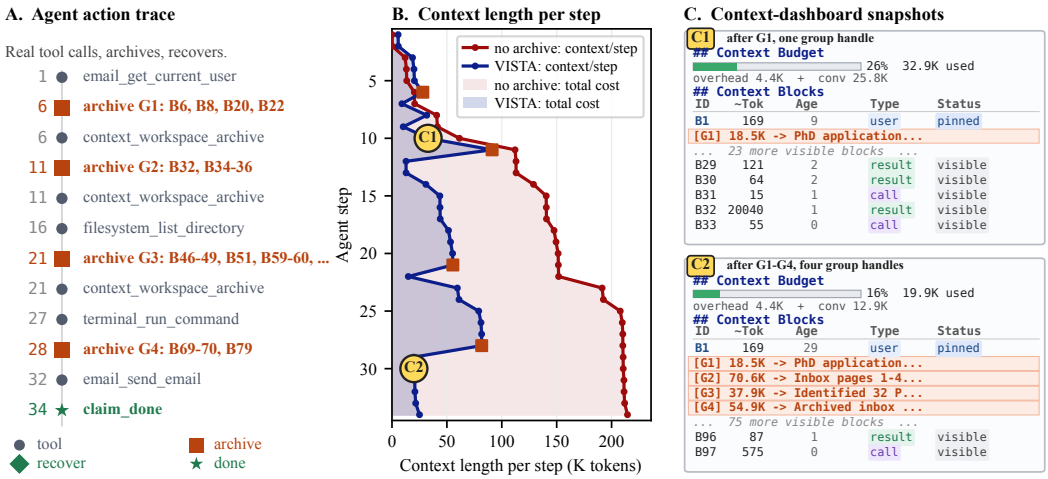


Figure 7: Case study trace. In one 128K LOCA-Bench run, VISTA archives large evidence, keeps the live context compact relative to a no-archive counterfactual, and recovers payloads when needed.

4 Analysis

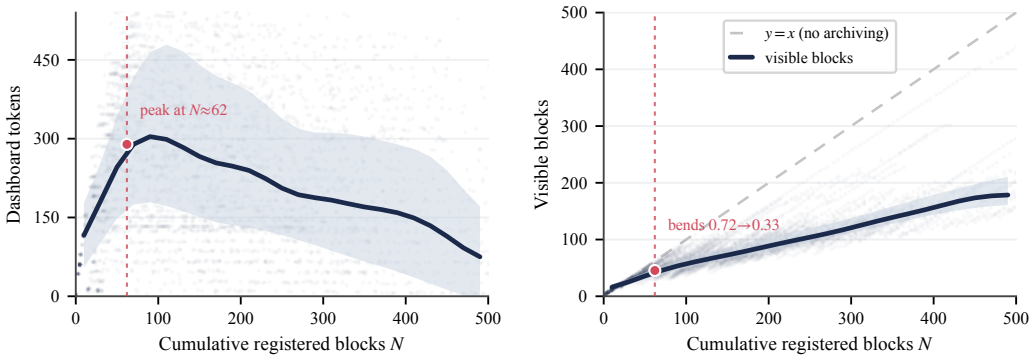
Does the perception gap actually exist? VISTA assumes an agent cannot read its own context state from the prompt, because size, recency, and remaining budget are runtime metadata rather than text. We test this directly at the first archive moment of real LOCA-Bench runs. We strip the dashboard and ask the backbone to report its own state along three independent probes, namely total transcript size, individual block size, and pairwise size comparison, scoring each against exact token counts across four open and closed backbones (Table 3). The gap is consistent and large. Without the dashboard every backbone misjudges size, with median relative error from 0.43 to 0.84 and estimates essentially uncorrelated with truth. Adding the factual ledger collapses size error to zero on all four models and lifts pairwise discrimination toward perfect. The effect is specific to token magnitude rather than transcript memory, and it holds on open and closed weights alike, so the intervention is a factual interface rather than a stronger prompt or a larger model. This is the empirical counterpart of the proprioceptive channel in Theorem 1, since the dashboard is what moves the agent from the size-blind regime toward full state observability.

	Total size		Block size		Pairwise	
Backbone	−dash	+dash	−dash	+dash	−dash	+dash
Claude-Sonnet-4.5	0.84	**0.00**	0.37	**0.02**	0.67	0.83
DeepSeek-V4-Pro	0.44	**0.00**	0.28	**0.00**	0.75	1.00
GLM-5	0.48	**0.00**	0.35	**0.00**	0.73	0.88
Gemini-3-Flash	0.43	**0.00**	0.24	**0.00**	0.68	1.00

Table 3: The perception gap is real and token-magnitude specific. Self-estimated context state with the dashboard stripped (−dash) versus present (+dash). Size is median relative error; pairwise is within-2× accuracy.

What does a rescued run look like? Figure 7 shows one 128K LOCA-Bench email-triage run. The dashboard marks a large inbox-export block as the biggest item, the agent archives it, keeps the live prompt below the no-archive counterfactual, and later reads the exact payload back for the final action. A matched baseline summarizes or clears the block and cannot restore the verbatim value. Across rescued tasks, this full archive-then-recover loop appears in 8 of the 16 cases where VISTA archives at least one block. In 13 other archive-containing runs outside the rescued subset, archiving mainly frees space without later recovery. Thus VISTA does not win by discarding old evidence; it moves evidence out of view while preserving an addressable recovery path. The advantage is clearest on long trajectories, where VISTA lowers active-context overhead while improving accuracy.

Does the dashboard scale? The dashboard adds tokens, so a fair worry is that it grows without bound as a trajectory registers more blocks. It does not. We measure dashboard cost on the three official BrowseComp-Plus runs (deepseek-v4-pro, W=12,288, B=163,840; the runs whose mean accuracy is the reported 58.0%), using the harness's per-turn accounting over all 30 queries per run. Each turn logs a fixed overhead of tool schemas plus system prompt (a constant 492 tokens across



(a) Dashboard cost vs. blocks **(b) Visible vs. total blocks**

Figure 8: Dashboard overhead is bounded by the active window, not the trajectory length (BrowseComp-Plus). **(a)** Net dashboard tokens (tool schemas and system prompt excluded) against cumulative registered blocks N: the cost peaks near $N{\approx}62$ and then declines. **(b)** The visible working set grows sublinearly in N (slope bends $0.72 \to 0.33$ at the same N), because archiving moves blocks into recoverable handles and out of the rendered ledger. Solid lines are binned medians; shaded bands are interquartile ranges.

every query, computed with the `cl100k_base` tokenizer) and the dashboard message; we subtract the fixed component to isolate the dashboard's own cost. Figure 8(a) shows that this net footprint is not linear in the trajectory length: it peaks near $N{\approx}62$ at roughly 340 tokens and then *declines* as N grows, because the dashboard renders the visible working set rather than the full history. Figure 8(b) explains why: the visible set grows sublinearly in the cumulative block count, with its slope bending from 0.72 to 0.33 at the same $N{\approx}62$, as archiving continually moves blocks out of the visible ledger into recoverable handles. The turning point in (a) coincides with the slope change in (b): once archiving keeps pace with new blocks, the visible set—and therefore the dashboard footprint—stops tracking N and is instead bounded by the active window B. Dashboard overhead is thus decoupled from total trajectory length.

5 Related Work

Prior work clarifies where context decisions sit and what state the agent can observe.

Context managed for the agent. One line keeps context decisions outside the model policy. Stale-observation masking, Demand Paging, AgentOS, and AgentSwing hide, page, or route context in the runtime [7, 12, 18, 44]; such systems may track size, age, or usage, but that state remains internal to the controller. Structured-eviction and cache-efficient managers such as Context Window Lifecycle and TokenPilot push this line further, replacing lossy summarization with deterministic, semantically aware pruning and cache-friendly ingestion to extend the working horizon at lower token cost [23, 38], yet they still decide what to evict on the agent's behalf. Memory and retrieval systems such as MemGPT, Mem0, SimpleMem, MR.Agent, PlugMem, BudgetMem, and SkillPro organize prior experience through virtual context, long-term memory, or retrieval/graph stores [5, 9, 15, 20, 22, 39, 45]. These systems provide useful storage or routing substrates, but the agent does not receive a per-block map of its active prompt. In Figure 1, this corresponds to context being managed *for* the agent: the runtime may discard stale observations, page records, or retrieve memories, but the model sees only the resulting prompt. It therefore cannot directly decide which visible evidence should stay, move out of view, or return later, nor can it trade a small but critical item against a large expendable tool result.

Self-managed context without state metadata. A second line makes context management part of the agent loop. Context as a Tool, Active Context Compression, ContextBudget, Context-Folding, and LongSeeker expose compression, folding, deletion, or routing actions, often through learned policies or specialized controllers [16, 17, 24, 25, 30]. This line is closest to our notion of self-management: the agent or controller can invoke context operations rather than passively waiting for truncation. However, these operations usually act on summaries, milestones, commits, or branches rather than on

a persistent block table. ContextBudget makes token pressure explicit, and Context-Folding gives branch and return actions, but they do not expose the VISTA-style state needed to target a precise oversized block while preserving a small exact detail. Other systems add hierarchy, compression guidance, agent-compatible managers, learned memory operations, or compact reasoning summaries [2, 8, 10, 11, 13, 26, 28, 29, 41, 42, 46], and PACE adapts memory granularity by predicting each item's relevance to the next action [27]. These works support our premise that context management is an agent-level decision, but their compressed representations are generally lossy and coarse relative to tool traces that contain exact identifiers, URLs, or table rows. VISTA instead isolates what runtime state the agent must perceive, then pairs that perception with block-level lossless archive and recovery.

Self-state awareness. Work on budget awareness, temporal blindness, and agent externalization studies signals that models cannot infer from prompt contents alone [1, 4, 14, 48]. Complementary probing work finds that agent-critical information such as plans is context-resident rather than persisted in hidden state, decaying once removed from the visible context [19], motivating an explicit self-description of working memory rather than assuming the model retains it. VISTA treats context state as such a signal and supplies it externally at inference time. This framing separates our contribution from simply enlarging the context window or improving summarizers: the missing information is not only more text, but a compact self-description of the agent's current working memory. In our setting, that self-description includes which blocks exist, how large and old they are, whether they are visible or archived, and whether they can be recovered exactly. This is the Figure 1 contrast: the agent does not merely see context, but also sees a compact state view over that context. We evaluate this view mainly on LOCA-Bench, which stresses online tool agents under controllable context growth [43], and use AMA-Bench as a memory-oriented transfer point [47].

6 Conclusion

We argued that frontier LLMs are proprioceptively blind to their own context: they cannot perceive how large, old, or used each piece of working memory is. VISTA addresses this with a runtime dashboard plus lossless archive and recovery, giving the agent context-state information it can act on without destroying evidence. With no training, the same interface outperforms ReAct, deletion, masking, compaction, and Claude Code on LOCA-Bench and transfers across four backbones. The result suggests that some agent capabilities are elicited by making hidden runtime state perceptible, positioning the interface as complementary to post-training rather than a replacement.

References

[1] Christopher Ackerman. Evidence for limited metacognition in LLMs, 2026. URL `https://arxiv.org/abs/2509.21545`.

[2] Milad Aghajohari, Kamran Chitsaz, Amirhossein Kazemnejad, Sarath Chandar, Alessandro Sordoni, Aaron Courville, and Siva Reddy. The markovian thinker: Architecture-agnostic linear scaling of reasoning. *arXiv preprint arXiv:2510.06557*, 2025.

[3] Zijian Chen, Xueguang Ma, Shengyao Zhuang, Ping Nie, Kai Zou, Andrew Liu, Joshua Green, Kshama Patel, Ruoxi Meng, Mingyi Su, Sahel Sharifymoghaddam, Yanxi Li, Haoran Hong, Xinyu Shi, Xuye Liu, Nandan Thakur, Crystina Zhang, Luyu Gao, Wenhu Chen, and Jimmy Lin. Browsecomp-plus: A more fair and transparent evaluation benchmark of deep-research agent, 2025. URL `https://arxiv.org/abs/2508.06600`.

[4] Yize Cheng, Arshia Soltani Moakhar, Chenrui Fan, Parsa Hosseini, Kazem Faghih, Zahra Sodagar, Wenxiao Wang, and Soheil Feizi. Your LLM agents are temporally blind: The misalignment between tool use decisions and human time perception. *arXiv preprint arXiv:2510.23853*, 2025.

[5] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

[6] Xilin Dai, Zhijian Xu, Wanxu Cai, and Qiang Xu. From samples to scenarios: A new paradigm for probabilistic forecasting. In *The Fourteenth International Conference on Learning Representations*, 2026.

[7] Zhaopeng Feng, Liangcai Su, Zhen Zhang, Xinyu Wang, Xiaotian Zhang, Xiaobin Wang, Runnan Fang, Qi Zhang, Baixuan Li, Shihao Cai, et al. AgentSwing: Adaptive parallel context management routing for long-horizon web agents. *arXiv preprint arXiv:2603.27490*, 2026.

[8] Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, and Ping Luo. Hiagent: Hierarchical working memory management for solving long-horizon agent tasks with large language model. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 32779–32798, 2025.

[9] Shuo Ji, Yibo Li, and Bryan Hooi. Memory is reconstructed, not retrieved: Graph memory for llm agents, 2026. URL `https://arxiv.org/abs/2606.06036`.

[10] Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. Acon: Optimizing context compression for long-horizon llm agents. *arXiv preprint arXiv:2510.00615*, 2025.

[11] Vasilis Kontonis, Yuchen Zeng, Shivam Garg, Lingjiao Chen, Hao Tang, Ziyan Wang, Ahmed Awadallah, Eric Horvitz, John Langford, and Dimitris Papailiopoulos. Memento: Teaching llms to manage their own context. *arXiv preprint arXiv:2604.09852*, 2026.

[12] ChengYou Li, XiaoDong Liu, XiangBao Meng, and XinYu Zhao. Architecting agentos: From token-level context to emergent system-level intelligence, 2026. URL `https://arxiv.org/abs/2602.20934`.

[13] Jiaqing Liang, Jinyi Han, Weijia Li, Xinyi Wang, Zhoujia Zhang, Zishang Jiang, Ying Liao, Tingyun Li, Ying Huang, Hao Shen, et al. GenericAgent: A token-efficient self-evolving llm agent via contextual information density maximization. *arXiv preprint arXiv:2604.17091*, 2026.

[14] Yuxiang Lin, Zihan Wang, Mengyang Liu, Yuxuan Shan, Longju Bai, Junyao Zhang, Xing Jin, Boshan Chen, Jinyan Su, Xingyao Wang, Jiaxin Pei, and Manling Li. Bagen: Are llm agents budget-aware?, 2026. URL `https://arxiv.org/abs/2606.00198`.

[15] Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. SimpleMem: Efficient lifelong memory for llm agents. *arXiv preprint arXiv:2601.02553*, 2026.

[16] Shukai Liu, Jian Yang, Bo Jiang, Yizhi Li, Jinyang Guo, Xianglong Liu, and Bryan Dai. Context as a tool: Context management for long-horizon swe-agents. *arXiv preprint arXiv:2512.22087*, 2025.

[17] Yijun Lu, Rui Ye, Yuwen Du, Jiajun Wang, Songhua Liu, and Siheng Chen. Longseeker: Elastic context orchestration for long-horizon search agents. *arXiv preprint arXiv:2605.05191*, 2026.

[18] Tony Mason. The missing memory hierarchy: Demand paging for llm context windows, 2026. URL `https://arxiv.org/abs/2603.09023`.

[19] Aman Mehta and Anupam Datta. Plans don't persist: Why context management is load bearing for llm agents. *arXiv preprint arXiv:2606.22953*, 2026.

[20] Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. Skill-pro: Learning reusable skills from experience via non-parametric ppo for llm agents, 2026. URL `https://arxiv.org/abs/2602.01869`.

[21] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A benchmark for general ai assistants. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.

[22] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. MemGPT: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[23] Andrew Semenov and Svyatoslav Dorofeev. Beyond compaction: Structured context eviction for long-horizon agents. *arXiv preprint arXiv:2606.11213*, 2026.

[24] Weiwei Sun, Miao Lu, Zhan Ling, Kang Liu, Xuesong Yao, Yiming Yang, and Jiecao Chen. Scaling long-horizon llm agent via context-folding. *arXiv preprint arXiv:2510.11967*, 2025.

[25] Nikhil Verma. Active context compression: Autonomous memory management in llm agents. *arXiv preprint arXiv:2601.07190*, 2026.

[26] Zhenting Wang, Huancheng Chen, Jiayun Wang, and Wei Wei. Memex (rl): Scaling long-horizon llm agents via indexed experience memory. *arXiv preprint arXiv:2603.04257*, 2026.

[27] Lei Wei, Xiao Peng, Guannan Zhang, Chenhao Jiang, Hongyu Li, Lanbo Lin, Yuanwu Xu, Jiayao Liu, Kesu Wang, Bin Wang, et al. Pace: Predictive adaptive context extraction for long-horizon llm agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27184–27199, 2026.

[28] Ian Wu, Yuxiao Qu, Amrith Setlur, and Aviral Kumar. Reasoning cache: Continual improvement over long horizons via short-horizon rl. *arXiv preprint arXiv:2602.03773*, 2026.

[29] Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Xinmiao Yu, Dingchu Zhang, Yong Jiang, et al. Resum: Unlocking long-horizon search intelligence via context summarization. *arXiv preprint arXiv:2509.13313*, 2025.

[30] Yong Wu, YanZhao Zheng, TianZe Xu, ZhenTao Zhang, YuanQiang Yu, JiHuai Zhu, Chao Ma, BinBin Lin, BaoHua Dong, HangCheng Zhu, et al. Contextbudget: Budget-aware context management for long-horizon search agents. *arXiv preprint arXiv:2604.01664*, 2026.

[31] Binyan Xu, Xilin Dai, Di Tang, and Kehuan Zhang. One surrogate to fool them all: Universal, transferable, and targeted adversarial attacks with clip. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, pages 3087–3101, 2025.

[32] Binyan Xu, Fan Yang, Xilin Dai, Di Tang, and Kehuan Zhang. Clip-guided backdoor defense through entropy-based poisoned dataset separation. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 7415–7423, 2025.

[33] Binyan Xu, Xilin Dai, Fan Yang, and Kehuan Zhang. When agent automation becomes profitable: Quantifying and insuring autonomous ai risk through trace-economic underwriting. *arXiv preprint arXiv:2606.16465*, 2026.

[34] Binyan Xu, Xilin Dai, and Kehuan Zhang. Contextual agentic memory is a memo, not true memory. *arXiv preprint arXiv:2604.27707*, 2026.

[35] Binyan Xu, Dong Fang, Haitao Li, and Kehuan Zhang. From multi-agent to single-agent: When is skill distillation beneficial? *arXiv preprint arXiv:2604.01608*, 2026.

[36] Binyan Xu, Fan Yang, Xilin Dai, Di Tang, and Kehuan Zhang. From internal diagnosis to external auditing: A vlm-driven paradigm for online test-time backdoor defense. *arXiv preprint arXiv:2601.19448*, 2026.

[37] Binyan Xu, Fan Yang, Di Tang, Xilin Dai, and Kehuan Zhang. Breaking the stealth-potency trade-off in clean-image backdoors with generative trigger optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 27197–27205, 2026.

[38] Buqiang Xu, Zirui Xue, Dianmou Chen, Chenyang Fu, Chiyu Wu, Caiying Huang, Chen Jiang, Jizhan Fang, Xinle Deng, Yijun Chen, et al. Tokenpilot: Cache-efficient context management for llm agents. *arXiv preprint arXiv:2606.17016*, 2026.

[39] Ke Yang, Zixi Chen, Xuan He, Jize Jiang, Michel Galley, Chenglong Wang, Jianfeng Gao, Jiawei Han, and ChengXiang Zhai. PlugMem: A task-agnostic plugin memory module for llm agents, 2026. URL `https://arxiv.org/abs/2603.03296`.

[40] Howard Yen, Ashwin Paranjape, Mengzhou Xia, Thejas Venkatesh, Jack Hessel, Danqi Chen, and Yuhao Zhang. Lost in the maze: Overcoming context limitations in long-horizon agentic search. *arXiv preprint arXiv:2510.18939*, 2025.

[41] Lu Yi, Runlin Lei, Liuyi Yao, Yuexiang Xie, Yuyang Li, Wenhao Zhang, Zhewei Wei, Yaliang Li, and Jian-Yun Nie. Learning agent-compatible context management for long-horizon tasks. *arXiv preprint arXiv:2605.30785*, 2026.

[42] Yi Yu, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. Agentic memory: Learning unified long-term and short-term memory management for large language model agents. *arXiv preprint arXiv:2601.01885*, 2026.

[43] Weihao Zeng, Yuzhen Huang, and Junxian He. LOCA-bench: Benchmarking language agents under controllable and extreme context growth. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026.

[44] Haoxiang Zhang, Qixin Xu, Zhuofeng Li, Lei Zhang, Pengcheng Jiang, Yu Zhang, and Julian McAuley. Masking stale observations helps search agents–until it doesn't: A regime map and its mechanism. *arXiv preprint arXiv:2606.00408*, 2026.

[45] Haozhen Zhang, Haodong Yue, Tao Feng, Quanyu Long, Jianzhu Bao, Bowen Jin, Weizhi Zhang, Xiao Li, Jiaxuan You, Chengwei Qin, and Wenya Wang. Learning query-aware budget-tier routing for runtime agent memory, 2026. URL https://arxiv.org/abs/2602.06025.

[46] Yuxiang Zhang, Jiangming Shu, Ye Ma, Xueyuan Lin, Shangxi Wu, and Jitao Sang. Memory as action: Autonomous context curation for long-horizon agentic tasks. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 19149–19164, 2026.

[47] Yujie Zhao, Boqin Yuan, Junbo Huang, Haocheng Yuan, Zhongming Yu, Haozhou Xu, Lanxiang Hu, Abhilash Shankarampeta, Zimeng Huang, Wentao Ni, et al. AMA-bench: Evaluating long-horizon memory for agentic applications. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026.

[48] Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, Congming Zheng, Jiachen Zhu, Zeyu Zheng, Zhuosheng Zhang, Xingyu Lou, Changwang Zhang, Zhihui Fu, Jun Wang, Weiwen Liu, Jianghao Lin, and Weinan Zhang. Externalization in llm agents: A unified review of memory, skills, protocols and harness engineering, 2026. URL https://arxiv.org/abs/2604.08224.

A Proof of Proposition 1

This is Fano's inequality in *reconstruction* mode: a budget-limited state cannot reproduce more blocks than it has bits for. We restate the setting. The history holds N blocks $X_1, \ldots, X_N$, each an independent string of k uniformly random bits, so $H(X_i) = k$ and the X_i are mutually independent. A non-recovering method holds a pre-reveal in-prompt state R with $H(R) \le B$. The state R is a function of the blocks and the method's internal randomness, formed before the query index $i^\star$ is revealed, and $i^\star$ is drawn uniformly on $\{1, \ldots, N\}$ independently of everything else. After the reveal the method outputs a guess $g(R, i^\star)$, and it is correct when $g(R, i^\star) = X_{i^\star}$.

Let $P_e^{(i)} = \Pr[g(R, i) \neq X_i]$ and let the reported success probability be $1 - P_e$ with $P_e = \frac{1}{N}\sum_{i=1}^{N} P_e^{(i)}$, the average over the uniform $i^\star$.

Step 1: per-block Fano bound. Fix a block i. Since X_i is uniform on an alphabet of size 2^k, Fano's inequality applied to the estimator $g(R, i)$ gives

$$H(X_i \mid R) \;\le\; H_b\Big(P_e^{(i)}\Big) + P_e^{(i)} \log_2(2^k - 1) \;\le\; 1 + P_e^{(i)}\, k,$$

where H_b is the binary entropy function, bounded by 1.

Step 2: independence couples the blocks to a budget. Because the X_i are mutually independent, $H(X_{1:N}) = \sum_i H(X_i)$, and subadditivity of conditional entropy gives $H(X_{1:N} \mid R) \le \sum_i H(X_i \mid R)$. Hence

$$\begin{aligned}\sum_{i=1}^{N} I(X_i; R) &= \sum_i \big(H(X_i) - H(X_i \mid R)\big) \\ &\le H(X_{1:N}) - H(X_{1:N} \mid R) \\ &= I(X_{1:N}; R) \;\le\; H(R) \;\le\; B.\end{aligned}$$

Step 3: combine. Using $I(X_i; R) = k - H(X_i \mid R) \ge k - 1 - P_e^{(i)} k$ from Step 1 and summing,

$$\begin{aligned}B &\ge \sum_{i=1}^{N} I(X_i; R) \\ &\ge \sum_{i=1}^{N} \big(k - 1 - P_e^{(i)} k\big) = Nk - N - k\sum_{i=1}^{N} P_e^{(i)}.\end{aligned}$$

Dividing by Nk and using $P_e = \frac{1}{N}\sum_i P_e^{(i)}$,

$$P_e \ge 1 - \frac{1}{k} - \frac{B}{Nk},$$

$$\Pr[\text{correct}] = 1 - P_e \le \frac{B}{Nk} + \frac{1}{k}.$$

VISTA attains probability one. VISTA writes each block to external storage as an exact transcript and keeps only a compact handle in the prompt. The pre-reveal prompt holds the instruction and N handles, whose size is $O(N \log N)$ rather than $O(Nk)$. Once $i^\star$ is revealed, the agent reads payload $i^\star$ and recovers $X_{i^\star}$ byte for byte. Whenever the instruction, the handles, and one recovered block fit within B, the method emits $X_{i^\star}$ exactly, so its success probability is 1.

Asymptotic separation. The gap statement requires a regime in which VISTA *stays feasible* while the lossy bound vanishes. Feasibility needs the N handles plus one recovered block to fit, i.e. $B \ge c_0 N \log_2 N + k$ for the constant c_0 set by the handle encoding; the lossy bound $\frac{B}{Nk} + \frac{1}{k}$ vanishes when $Nk/B \to \infty$ and $k \to \infty$. Both hold, for example, at $k = \lceil \sqrt{N} \rceil$ and $B = cN \log_2 N$ with $c > c_0$: then B dominates the handle cost, so VISTA is feasible and stays at success 1, while $\frac{B}{Nk} + \frac{1}{k} = \Theta\big(\frac{\log N}{\sqrt{N}}\big) \to 0$, so the lossy success probability tends to 0 and the gap tends to 1 as $N \to \infty$. A *fixed* B does not exhibit this separation, because then VISTA's own $O(N \log N)$ handle table eventually violates the budget. The separation is thus a statement about the growth rate of the raw evidence Nk relative to a budget B that grows only fast enough to index it. □

B Proof of Proposition 2

We locate the dashboard-free endpoint of Def. 2. Conditioned on $J^\star = j$, the observation $W = (W_1, \dots, W_n)$ is Gaussian with independent coordinates, $W_i \sim \mathcal{N}(m_i^{(j)}, \sigma^2)$, where the mean vector $m^{(j)}$ has $m_j^{(j)} = \ln L$ (the bulky block) and $m_i^{(j)} = \ln \ell$ for $i \neq j$. Let P_j denote this conditional law and $\bar{P} = \frac{1}{n}\sum_k P_k$ the mixture.

Step 1: mutual information as mixture KL. For $J^\star$ uniform, a standard identity gives

$$I(J^\star; W) \;=\; \frac{1}{n}\sum_{j=1}^{n} \mathrm{KL}\big(P_j \,\|\, \bar{P}\big)\,.$$

Because $\mathrm{KL}(P \,\|\, \cdot)$ is convex in its second argument and $\bar{P} = \frac{1}{n}\sum_k P_k$, Jensen gives $\mathrm{KL}(P_j \| \bar{P}) \le \frac{1}{n}\sum_k \mathrm{KL}(P_j \| P_k)$, hence

$$I(J^\star; W) \;\le\; \frac{1}{n^2}\sum_{j=1}^{n}\sum_{k=1}^{n} \mathrm{KL}(P_j \,\|\, P_k)\,.$$

Step 2: pairwise Gaussian KL. P_j and P_k are Gaussians with the same covariance $\sigma^2\mathbf{I}$, so $\mathrm{KL}(P_j \| P_k) = \frac{1}{2\sigma^2}\|m^{(j)} - m^{(k)}\|_2^2$. For $j \neq k$ the mean difference is nonzero only in coordinates j and k: coordinate j contributes $\ln L - \ln \ell = \ln \kappa$ and coordinate k contributes $\ln \ell - \ln L = -\ln \kappa$, so $\|m^{(j)} - m^{(k)}\|_2^2 = 2(\ln \kappa)^2$ and

$$\mathrm{KL}(P_j \| P_k) = \frac{(\ln \kappa)^2}{\sigma^2} \quad (j \neq k), \qquad \mathrm{KL}(P_j \| P_j) = 0.$$

Step 3: combine. There are $n(n-1)$ ordered pairs with $j \neq k$, so

$$I(J^\star; W) \;\le\; \frac{1}{n^2}\, n(n-1)\, \frac{(\ln \kappa)^2}{\sigma^2} \;\le\; \frac{(\ln \kappa)^2}{\sigma^2} \text{ nats} \;=\; \frac{(\log_2 \kappa)^2}{\sigma^2}\, \ln 2 \text{ bits},$$

the last equality using $\ln \kappa = (\ln 2)\log_2 \kappa$. The bound depends only on the log size-ratio $\log_2 \kappa$ and the perception noise σ, and is constant in the workspace size n. This is the dashboard-free endpoint: it sits at a fixed rate while the full ledger sits at $\log_2 n$. By Theorem 1, holding $\mathbb{E}[Z] \le \delta\kappa$ needs $(1 + o(1))\log_2(n/\kappa)$ bits, so for any fixed σ a large enough workspace pushes the content channel below the make-room threshold and the ledger becomes necessary. □

C Proof of Theorem 1

The proof has four parts: (A) the list-Fano lemma; (B) a reduction that turns the make-room recovery cost into a list-decoding error, converting the information budget I into a lower bound on $\mathbb{E}[Z]$; (C) the closed-form tradeoff and its corollaries (endpoints, threshold, price); and (D) a matching achievability construction. Throughout, $J^\star$ is uniform on $[n] := \{1, \dots, n\}$, $\kappa = L/\ell \in \{2, \dots, n-1\}$ is an integer, and $\nu := \log_2 \frac{n-\kappa}{\kappa}$.

(A) List-Fano lemma (proof of Lemma 1). Let $\mathcal{L} = \mathcal{L}(Y, U)$ be a list of size m and $E := \mathbf{1}[J^\star \notin \mathcal{L}]$, so $\Pr[E{=}0] = P_c$. Expand $H(J^\star, E \mid Y, U)$ two ways. Since E is a function of $(J^\star, Y, U)$, $H(J^\star, E \mid Y, U) = H(J^\star \mid Y, U)$. Also

$$H(J^\star, E \mid Y, U) = H(E \mid Y, U) + H(J^\star \mid E, Y, U) \le 1 + H(J^\star \mid E, Y, U).$$

Condition on E: given $E{=}0$, $J^\star$ lies in a set of size $\le m$, so $H(J^\star \mid E{=}0, Y, U) \le \log_2 m$; given $E{=}1$, $J^\star$ lies in the complement of size $\le n - m$ (recall $\mathcal{L}$ is (Y, U)-measurable), so $H(J^\star \mid E{=}1, Y, U) \le \log_2(n - m)$. Averaging with weights $P_c, 1 - P_c$ and using $H(J^\star \mid Y, U) = H(J^\star) - I(J^\star; Y, U) = \log_2 n - I$ (as $U \perp J^\star$, $I(J^\star; Y, U) = I(J^\star; Y) = I$),

$$\log_2 n - I \;\le\; 1 + P_c \log_2 m + (1 - P_c)\log_2(n - m),$$

which is Lemma 1. □

(B) Reduction: recovery cost lower-bounds via list decoding. By the "still-over" argument (part (E) below), any admissible make-room policy is equivalent in cost to archiving blocks in some order $\pi = (\pi_1, \pi_2, \dots)$ that is measurable in (Y, U) and stopping at the first time the freed size reaches L. Because the bulky block frees L by itself while each load-bearing block frees only ℓ, the process stops exactly at step $T = \min(\tau, \kappa)$ where τ is the position of $J^\star$ in π: if $\tau \le \kappa$ it captures $J^\star$ and frees L; if $\tau > \kappa$ it has archived κ load-bearing blocks, freeing $\kappa\ell = L$, before reaching $J^\star$. The number of archived load-bearing blocks, which are exactly the ones that must later be recovered, is (5), $Z = \min(\tau - 1, \kappa)$.

Fix any integer $1 \le m \le \kappa$ and let $\mathcal{L}_m := \{\pi_1, \dots, \pi_m\}$, the first m probed blocks; this is a size-m, (Y, U)-measurable list. The key observation is

$$Z \ge m \iff \tau > m \iff J^\star \notin \mathcal{L}_m, \tag{6}$$

because $Z = \min(\tau - 1, \kappa) \ge m$ iff $\tau - 1 \ge m$ (using $m \le \kappa$) iff $J^\star$ is not among the first m probes. Hence, writing $P_c(m) := \Pr[J^\star \in \mathcal{L}_m]$, we have $\Pr[Z \ge m] = 1 - P_c(m)$.

Expectation via tail sum. Since $Z \in \{0, 1, \dots, \kappa\}$,

$$\mathbb{E}[Z] = \sum_{m=1}^{\kappa} \Pr[Z \ge m] = \sum_{m=1}^{\kappa} \big(1 - P_c(m)\big) = \kappa - \sum_{m=1}^{\kappa} P_c(m). \tag{7}$$

One-shot list bound. Apply Lemma 1 to the size-κ list $\mathcal{L}_\kappa$. With $m = \kappa$ and $n - \kappa \ge \kappa$ (so $\nu = \log_2 \frac{n-\kappa}{\kappa} \ge 0$),

$$\log_2 n - I \le 1 + P_c(\kappa) \log_2 \kappa + (1 - P_c(\kappa)) \log_2 (n - \kappa).$$

Rearranging, and using $\log_2 n \ge \log_2 (n - \kappa)$,

$$-(I + 1) \;\le\; \big(\log_2 (n - \kappa) - \log_2 n\big) + P_c(\kappa)\big(\log_2 \kappa - \log_2 (n - \kappa)\big) \;\le\; -P_c(\kappa)\, \nu,$$

so $P_c(\kappa) \le \frac{I+1}{\nu}$. Because $\mathcal{L}_m \subseteq \mathcal{L}_\kappa$ for $m \le \kappa$, $P_c(m) \le P_c(\kappa) \le \frac{I+1}{\nu}$, and plugging into (7),

$$\boxed{\mathbb{E}[Z] \;\ge\; \kappa - \kappa \frac{I+1}{\nu} = \kappa\Big(1 - \frac{I+1}{\nu}\Big).} \tag{8}$$

This is the bound in Theorem 1. (When the right side is negative the statement is vacuous, consistent with $Z \ge 0$.)

(C) Corollaries. *(i) Endpoints.* The size-aware ledger reveals the size vector, which identifies the unique block of size L, so $I = \log_2 n$; then the agent archives $\{J^\star\}$, freeing exactly L, giving $\tau = 1$, $Z = 0$, hence 0 recoveries and 0 recall errors. The content-only agent (no ledger) has $I = I_{\text{content}}$ bounded by Proposition 2, a constant in n, so (8) gives $\mathbb{E}[Z] \ge \kappa(1 - \frac{I_{\text{content}}+1}{\nu}) \to \kappa$ as $n/\kappa \to \infty$ (since $\nu \to \infty$ while I_{content} stays bounded). The pure size-blind case $I = 0$ is the special instance $\sigma \to \infty$.

(ii) Threshold. $\mathbb{E}[Z] \le \delta\kappa$ forces, via (8), $\kappa(1 - \frac{I+1}{\nu}) \le \delta\kappa$, i.e. $I \ge (1 - \delta)\nu - 1$.

(iii) Price. The right-hand floor $\kappa(1 - \frac{I+1}{\nu})$ is affine in I with slope $-\kappa/\nu$, so raising I by one bit lowers the guaranteed recovery floor by at most κ/ν; achieving a reduction of Δ round-trips therefore requires $I \ge \Delta\nu/\kappa$ bits.

From Z to recoveries and errors. Each archived load-bearing block is queried later; answering correctly requires recovering it, one round-trip each, so the expected number of recoveries is $\ge \mathbb{E}[Z]$. If recoveries are capped at r, at least $\mathbb{E}[Z] - r$ archived load-bearing blocks are never restored in expectation, each causing an exact-recall failure. All three size-aware costs are 0.

(D) Achievability (matching upper bound). Consider an I-bit *bucket ledger*: partition $[n]$ into $2^{\lfloor I \rfloor}$ contiguous buckets, each of size at most $b := \lceil n\, 2^{-\lfloor I \rfloor} \rceil$, and report the bucket Y containing $J^\star$; then $I(J^\star; Y) \le \log_2 2^{\lfloor I \rfloor} = \lfloor I \rfloor \le I$, so this is a rate-$\le I$ interface. The agent probes blocks inside the reported bucket in uniformly random order until the flag clears. The bulky block sits at a uniform position $P \in \{1, \dots, b'\}$ within its bucket of size $b' \le b$, and as in part (E) it frees L upon capture, so the number of load-bearing blocks archived is $\min(P - 1, \kappa) \le P - 1$, giving

$$\mathbb{E}[Z] \;\le\; \mathbb{E}[P - 1] = \frac{b' - 1}{2} \;\le\; \frac{b - 1}{2} \;\le\; \frac{n}{2}\, 2^{-\lfloor I \rfloor},$$

Method	Training-free	Model-agnostic	Agent-controlled	Exact recovery	Context dashboard
ReAct (append until truncation)	✓	✓	✗	✗	✗
Tool-result clearing	✓	✓	✗	✗	✗
Stale-observation masking	✓	✓	✗	✗	✗
Active Context Compression	✓	✓	✓	✗	✗
Skeleton compression	✓	✓	✗	✗	✗
Claude Code	✓	$\sim$	✓	$\sim$	✗
Learned compression (CAT, RL budget)	✗	✗	✓	✗	✗
Context-Folding	✗	✗	✓	✗	✗
LongSeeker	✗	✗	✓	✗	✗
GenericAgent	✗	✗	✓	✗	✗
VISTA (ours)	✓	✓	✓	✓	✓

Table 4: Method capability comparison. Capability summary against the design properties of Section 3; VISTA uniquely combines agent-facing state with exact recovery.

where the last step uses $\lceil x \rceil - 1 < x$. Thus $\mathbb{E}[Z] \le \delta\kappa$ whenever $\frac{n}{2}2^{-\lfloor I \rfloor} \le \delta\kappa$, i.e. $\lfloor I \rfloor \ge \log_2 \frac{n}{2\delta\kappa}$; and $I = \log_2 n$ gives buckets of size 1, hence $P = 1$, $Z = 0$. For fixed δ this sufficient rate is $\log_2(n/\kappa) - \log_2(2\delta) + O(1) = (1 + o(1))\nu$, matching the necessary rate $(1 - \delta)\nu - 1$ of (ii) up to the $(1 + o(1))$ factor as $n/\kappa \to \infty$. Hence the transition of $\mathbb{E}[Z]$ from $\Theta(\kappa)$ to $o(\kappa)$ occurs at $I^\star = (1 + o(1))\nu$, and $\nu = (1 + o(1)) \log_2(n/\kappa)$.

(E) Why archive-in-an-order is without loss, and adaptivity is useless. Two reductions were used above. First, any admissible archive *set* S of size m that exits overflow can be realized by an order that archives its members first; the recovered load-bearing count is $|S \setminus \{J^\star\}|$, matching Z in (5), so restricting to orders is without loss for the cost. Second, adaptivity through the binary flag adds no usable information: before the bulky block is archived, every archived prefix of $t \le \kappa - 1$ load-bearing blocks has freed $t\ell < L$, so the flag reads "over" deterministically and is independent of *which* blocks were chosen; after the bulky block is archived the freed size is $\ge L$ and the process stops. Thus the only $J^\star$-information available to the ordering is Y, exactly as assumed, and Lemma 1 applies to the (Y, U)-measurable prefix lists $\mathcal{L}_m$. This is why the lower bound (8) holds for all adaptive content-only ($I = I_{\text{content}}$) and rate-I policies alike. □

Remark 1 (Interpretation: dashboard bits versus recovery round-trips). Theorem 1 identifies a *phase transition in an information parameter*: recovery tools are wasted until the interface supplies $\Theta(\log(n/\kappa))$ bits about which block to evict, after which the recovery cost collapses, and the exchange rate is exactly κ/ν round-trips per bit. The no-dashboard agent perceives a genuine, noisy size signal from content, but Proposition 2 places that signal at $I_{\text{content}} \le (\ln 2)(\log_2 \kappa)^2/\sigma^2$ bits, constant in n, so it falls below the $\log_2(n/\kappa)$ make-room threshold once the workspace is large, while the full ledger reaches the $I = \log_2 n$ end. The empirical proxy for I_{content} is the model's size-*ranking* ability: the pairwise size-comparison accuracy of Appendix H measures how well content localizes the block to evict, and the dashboard drives it toward the $I = \log_2 n$ end (median relative size error 0.43–0.84 without the ledger, collapsing to 0 with it; Table 3). The $255/57$ vs. $69/105$ archive/retrieve split (Figure 6b) is the over-archive-under-retrieve signature the theorem predicts.

D Method Capability Comparison

Table 4 summarizes the evaluated baselines and the learned-compression family against the design properties of Section 3. It isolates the two properties that distinguish VISTA: an agent-facing context dashboard and byte-exact recovery of externalized evidence. The marks are a coarse capability summary rather than a performance claim, and partial marks reflect mechanisms that hold the property only in part, such as Claude Code, which keeps files on disk but still summarizes the active context.

E Implementation Details

This section reports the exact run configuration, the verbatim prompt the agent receives, the dashboard format, and the context tool definitions, so the setting can be reproduced without access to our harness.

Benchmark setting. LOCA-Bench evaluates online tool agents under controllable context growth: the agent must continue acting while earlier reasoning, tool calls, and observations remain in or are externalized from the working context. We evaluate 75 task configurations; unless otherwise stated, accuracy is solved tasks over all 75, with errors and timeouts counted as failures. AMA-Bench is used as a secondary generalization benchmark. Its episodes provide a completed trajectory and ask questions about past events, causal relations, and state changes. This is not the native online-control setting for VISTA; it tests whether the same context-management layer can be adapted into trajectory memory. We use the benchmark's two-stage memory interface. During memory construction, the completed trajectory is replayed step by step as a growing conversation: each action and observation becomes a workspace block, the future questions are hidden, and the replayed agent may archive exact payloads when the workspace budget becomes tight. During retrieval, VISTA assembles the resulting workspace, dashboard, construction events, and recoverable archive handles for the current question. Thus the AMA-Bench result evaluates a replayed VISTA workspace as offline memory [34], rather than simply placing the full trajectory in the model prompt.

Run configuration. VISTA is integrated into the LOCA-Bench harness and invoked as a strategy (`loca run -s self_managed`) with no training and no per-model tuning. The main results use `gemini-3-flash` at a 128K budget (`max-context-size` $= 128{,}000$); the cross-backbone runs reuse the same strategy unchanged on `claude-sonnet-4-5`, `deepseek-v4-pro` (open-weight), and `glm-5` (open-weight). Two flags define the full method: `SM_STRICT_LONG_CONTEXT=1` enforces a hard budget rather than a soft warning, and `SM_BETTER_DASHBOARD=1` selects the factual ledger dashboard below. The ablations toggle single flags from this base, for example `SM_DISABLE_ARCHIVE`, `SM_DISABLE_AGENT_ARCHIVE` (fixed archive policy), and `SM_ENABLE_STATE_BOARD` (status-board variant). Per-task timeout is 1800 seconds and reasoning effort is medium across all backbones.

Baseline definitions. We organize the LOCA-Bench baselines by who makes the keep-or-drop decision. Fixed external policies include ReAct, which appends until truncation; Tool-result Clearing, which removes old tool-result/tool-call pairs after the prompt crosses a threshold; and fixed stale masking, which masks old tool observations while preserving the assistant reasoning and tool-call skeleton. Agent-mediated baselines still reduce context irreversibly. SLIM [40], reproduced from its public release, periodically summarizes older context once the budget is exceeded. Active Context Compression [25] asks the agent to write and prune its own knowledge blocks. A structured-compression baseline preserves a compact skeleton of prior context, following the design of context-as-a-tool compressors [16]. Learned members of this family generally do not release trained checkpoints and, in many cases, do not release code, so we reproduce the training-free methods directly and follow the published inference-time design for the rest rather than a trained policy. Claude Code is the Claude Code command-line agent at the CLI release of May 6, 2026, included as a strong practical agent with mature tool-use and context-handling heuristics. These baselines cover deletion, masking, summarization, self-compression, and structured compression. None combines agent-facing context-state metadata with exact evidence recovery. On AMA-Bench, the EMem-style and Mem0-style rows are local adapters implemented for this harness and should be read as engineering baselines rather than official reproductions.

Baseline reproduction details. SLIM and Active Context Compression are faithful reproductions of training-free published methods, run with the procedure described by their authors and triggered at the same 128K budget used for every method. SLIM periodically summarizes older context once the budget is exceeded, and Active Context Compression runs the explore, write a knowledge block, then prune the raw history loop. The structured-compression baseline is inspired by context-as-a-tool compressors rather than a faithful reimplementation, since that method is learned and we run no trained policy. Across all baselines we vary only the context-management mechanism and hold the agent loop, tools, budget, backbone, and scoring fixed, so accuracy differences reflect the context policy rather than the surrounding harness.

Context-management protocol. The agent receives the following instruction block appended to the task prompt, together with a budget notice. It is identical across backbones.

Context-management protocol (verbatim)

```
CONTEXT MANAGEMENT PROTOCOL:
A <context_workspace_status> dashboard is shown every turn as a compact map of
context blocks. Use context tools only when clearly needed. Do not archive,
delete, or offload content solely because it is old, large, or listed in context
metadata; leave content visible when the context budget is sufficient. Large
payloads may be represented by placeholders; inspect originals only when needed.
Use ordinary file/terminal/python tools, source metadata, and any in-context
payload placeholders to inspect external evidence when details are needed. For
structured data or calculations, use the source file, source tool, or query
directly. Do not copy table, CSV, or JSON rows from the conversation into code.


You need to complete the task within the following context window size:
<budget:token_budget>128000</budget:token_budget>
```

Dashboard format. Each turn the harness injects a `<context_workspace_status>` block. It is a budget bar followed by one ledger row per block, with columns ID, approximate tokens, age (root-turn distance, where `0r` is newest), type, compression level, parent, and status (`visible`, `pinned`, `archived`, or `offloaded_placeholder`). The instance below shows the same compact column subset used in Figures 1 and 2; the full renderer additionally prints the compression Level and Parent columns described above.

Dashboard instance shown to the agent

```
## Context Budget
[############--------] 62%  (~79,400 / 128,000 tokens)
  overhead ~6,200 | conversation ~71,900 | dashboard ~1,300

## Context Blocks  (Age = root-turn distance; 0r newest)
ID      ~Tok   Age  Type               Status
-------------------------------------------------
B1       120   8r  user_message       pinned
B2    18,400   7r  tool_call          archived
B3     2,150   6r  assistant_message  visible
B5    14,800   3r  tool_call          visible
B6     9,300   2r  tool_call          visible
B9     1,070   0r  assistant_message  visible
```

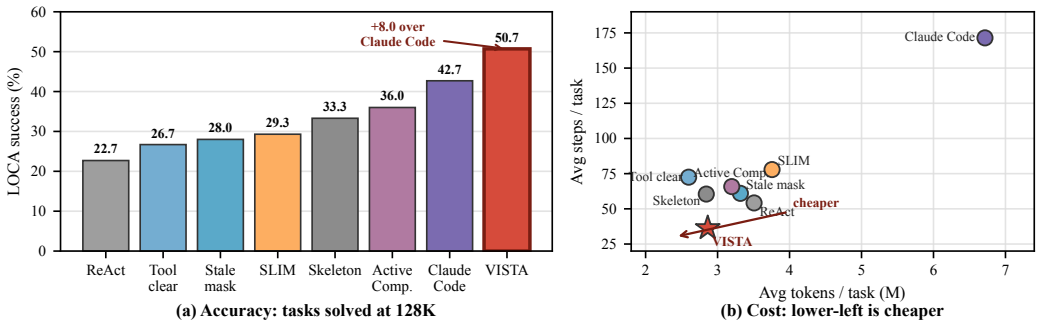
Context tool definitions. The agent acts on the workspace with two tools. Archiving replaces a block with a compact handle and returns the payload file path; the agent recovers byte-exact content by reading that path with ordinary file or terminal tools, so recovery is a normal read rather than a dedicated decompressor.

Context tool definitions (docstrings)

```
context_workspace_archive(block_id: str, replacement: str = "") -> str
  Replace one or more blocks with compact indexes. The original content is
  stored externally as a payload file. Operations are block-level: only listed
  block IDs are archived. Returns the payload file path for later recovery.
    block_id:    Block IDs, ranges, or group IDs, e.g. "B3", "B3,B4",
                 "B10-B20", or "G2".
    replacement: Short index text for the archived block(s).

context_workspace_delete(block_id: str, reason: str) -> str
  Permanently remove one or more blocks. Deleted content cannot be recovered.
    block_id: Block IDs, ranges, or group IDs.
    reason:   Short reason why the content has no future task value.
```

Large tool results are stored as external transcript payloads with compact placeholders. These payloads record what a tool returned to the model, not a complete source database; if a transcript is truncated or paginated, the agent must query the original source tool for complete data.



(a) Accuracy: tasks solved at 128K **(b) Cost: lower-left is cheaper**

Figure 9: Expanded LOCA result at 128K. Tasks solved, tokens, and steps for the main LOCA-Bench comparison.

Family	Method	Correct	Acc.	Timeout	Error	Steps	Tokens	Mgmt. events	Notes
No CM	ReAct	17	22.7	0	0	54.6	3.51M	636 trims	full 75
Deletion	Tool-result Clearing	20	26.7	0	0	72.9	2.60M	1,968 clears	full 75
Masking	Fixed stale masking	21	28.0	–	–	61.0	3.32M	–	full 75
Summary	SLIM	22	29.3	–	–	77.9	3.76M	–	full 75
Self-compression	Active Context Compression	27	36.0	–	–	65.8	3.20M	–	full 75
Structured compression	Skeleton compression	25	33.3	–	–	60.5	2.84M	–	full 75
Agent CLI	Claude Code	32	42.7	0	22	171.5	6.72M	–	full 75
Ours	VISTA	38	50.7	20	0	36.4	2.86M	69 archive / 105 read	full 75

Table 5: Dense LOCA run ledger. 128K main comparison with execution cost and method-specific context-management events.

Comparison	Both	Base.	VISTA	Neither
Fixed stale masking	18	3	20	34
SLIM	17	5	21	32
Active Context Compression	20	7	18	30
Claude Code	25	7	13	30

Table 6: Outcome transitions. Pairwise 128K LOCA-Bench split against each baseline: both solve, baseline-only, VISTA-only, and neither.

F Evaluation Details

For LOCA-Bench, we use the independently released public task suite and evaluation protocol without modification. It contains 75 online tool-task configurations with controllable context growth, where prior reasoning, tool calls, and observations accumulate until context management becomes central. We report task success, count errors and timeouts as incorrect, and compute average steps and tokens over task rows present in each run log. These cost values therefore describe observed execution cost rather than normalized cost conditional on success. They also do not model downstream economic exposure or insurance-style trace risk [33], online auditing [36], or probabilistic forecasting settings that turn samples into scenarios [6]. We additionally log archive and recover/read events for VISTA variants.

Figure 9 and Table 5 expand the 128K LOCA-Bench comparison, Table 6 gives the pairwise rescued-task split, and Table 7 gives exact counts for the context-growth sweep in Figure 3.

For BrowseComp-Plus, we evaluate deep-research retrieval with DeepSeek-V4-Pro on an $N{=}150$ subset and report judged Pass@1 with one sampled answer. The agent searches a fixed, non-adversarial corpus [31], so evidence is scattered across retrieved passages and the transcript grows through repeated retrieval. To expose context management, we use a deliberately tight active window ($W{=}12$K tokens per call) and total budget ($B{=}160$K), chosen so early evidence can be evicted before synthesis.

For GAIA, we use a fixed random 165-question subset of the public validation split. We preserve the original question text and attached files, require the official `FINAL ANSWER:` format, and score

Method	Setting	Success	Avg. steps	Avg. tok.	Notes
VISTA	8K	82.7	16.7	0.44M	complete
VISTA	16K	84.0	16.2	0.52M	complete
VISTA	32K	70.7	17.4	0.87M	complete
VISTA	64K	61.3	21.2	1.38M	complete
VISTA	96K	57.3	29.7	2.32M	complete
VISTA	128K	50.7	36.4	2.86M	complete
VISTA	256K	32.0	43.1	3.51M	complete
ReAct	8K	84.0	27.9	0.68M	complete
ReAct	16K	74.7	24.9	0.80M	complete
ReAct	32K	65.3	25.3	1.02M	complete
ReAct	64K	52.0	29.6	1.70M	complete
ReAct	96K	36.0	39.5	2.79M	complete
ReAct	128K	22.7	54.2	3.51M	complete
ReAct	256K	12.0	82.6	5.80M	complete

Table 7: Observed pressure sweep. Matched VISTA and ReAct runs over the full 75-task suite.

	Embodied		Game		OpenQA		Software		Text2SQL		Web	
Method	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1
AMA	0.678	0.489	0.814	0.407	0.817	0.249	0.565	0.150	0.838	0.395	0.782	0.272
VISTA	0.683	0.525	0.747	0.443	0.731	0.386	0.567	0.238	0.846	0.384	0.763	0.341

Table 8: AMA domain breakdown. By-domain accuracy and F1 for AMA and the VISTA trajectory-memory adaptation.

Case	Baselines failed	VISTA steps	Archive/read evidence	Evaluation signal
NHL B2B schedule analysis	ReAct, Tool-clear, SLIM	64	5 archive calls; 17 payload reads/uses	CSV and Google Sheet correct; HA/AH/HH/AA counts verified
WooCommerce low-selling products	ReAct, Claude-style, Tool-clear, SLIM	38	4 archive calls; 3 payload reads/uses	Correct products moved; subscriber emails sent
Canvas final-exam schedule	ReAct, Claude-style, Tool-clear, SLIM	17	1 archive call for course announcements	Final Excel schedule accepted
NLP course reminders	ReAct, Claude-style, Tool-clear, SLIM	31	1 archive call for large roster table	Correct students emailed; dropped/submitted students excluded

Table 9: Rescued-task case studies. Tasks VISTA solves where multiple baselines fail, with archive and payload-use counts.

with quasi-exact match. All methods use DeepSeek-V4-Pro, real web/search/file tools, W=12K, and B=80K.

For AMA-Bench, each of the 208 episodes contains 12 open-ended questions, giving 2496 judged QA pairs. We report judge accuracy and token-level F1 following the benchmark harness. Runtime per episode is measured for generation; judge time is reported separately in the analysis files.

G Limitations

VISTA supplies the missing proprioceptive signals, but it does not guarantee the agent uses them well. A model can still misread the dashboard, archive evidence it later needs, or recover a payload too late. The elicitation view also predicts a floor: a model with little latent context-management skill has little for the interface to unlock, and GLM-5 gains least. We test four backbones; mapping the low-capability end of this curve remains open. We also do not test poisoned external evidence [32] or generative trigger settings [37].

VISTA is complementary to post-training rather than an alternative to it. Training improves what an agent does with context-state information, while the dashboard supplies information that is absent from the prompt. We do not combine VISTA with post-training, richer metadata such as predicted relevance, or learned context-management policies such as learned compression managers or memory-action policies [41, 42, 46]; nor do we test whether skills distilled from multi-agent systems change how a single agent uses the dashboard [35]. Finally, the EMem-style and Mem0-style AMA-Bench rows are local adapters rather than official implementations, so they support diagnosis but are not final claims against those systems; AMA-Bench remains a transfer test rather than a primary benchmark.

H Proprioceptive-Blindness Diagnostic

This appendix documents the diagnostic behind Table 3. The goal is to measure directly whether a backbone can read its own context state, separating perception from skill.

Data. We anchor on the first archive event of each real LOCA-Bench run, the moment the agent itself decided to externalize content. We take the accumulated transcript just before that call as the snapshot, treating each message as one block. The runtime dashboard is not persisted in the transcript, so the stored messages are already free of the live ledger; we additionally strip the three injected artifacts that would leak state, namely the context-management protocol header, the hard-limit rejection notices that print token counts, and archived-block placeholders. A scan over all anchored snapshots confirms no residual token, budget, or usage strings remain, and there is no per-block usage annotation. Twenty-nine runs contain an archive; we cap each snapshot at 100K tokens by dropping trailing blocks so it fits every backbone window, and compute ground truth with the same tokenizer used by the harness.

Questions and conditions. We ask four quantities, each in its own request so the measurements stay independent. *Total size*: estimate the token count of the whole transcript. *Block size*: estimate the token count of four sampled blocks. *Pairwise*: for sampled block pairs, say which is larger, reported on the hard subset within $2\times$ in true size. *Recency*: a quoted passage is shown and the model states how many model turns ago it appeared; for this question the transcript is rendered without block identifiers so the model is not handed an ordinal index, and the passage is verified unique. The $-$*dash* condition shows the cleaned transcript only; the $+$*dash* condition prepends the factual ledger (the columns of the implementation-details appendix). Size answers are scored as median relative error and pairwise as accuracy against the larger block. We run Gemini-3-Flash, Claude-Sonnet-4.5, DeepSeek-V4-Pro, and GLM-5 with greedy decoding. Claude-Sonnet-4.5 returns valid structured output slightly less often than the other three, but the qualitative gap and its closure with the dashboard hold for every backbone.

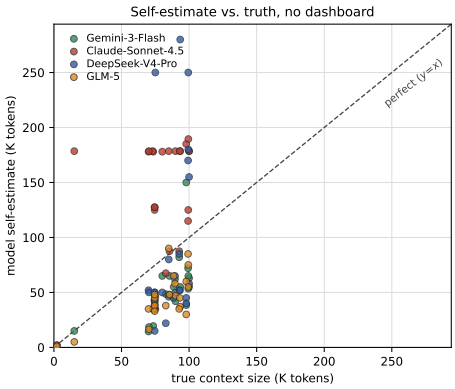


Figure 10: Proprioceptive-blindness diagnostic. Without the dashboard, self-estimated context size is poorly calibrated; the factual ledger closes the gap.